



Cairo University Racing Team

Formula Student / FSAE®

Software Development Team

(Fullstack - Technical Task)

Season 26-27

Team Project Management System

System Description

The system is designed to help teams organize their projects, manage tasks, and track the progress of their work. The platform allows users to create and manage projects, break projects down into tasks, assign tasks to team members, and track their status and priority.

Core Entities

Users

- Create an account and log in.
- Manage their profile.
- Participate in projects and tasks.

Projects

- Create a project with a name and description.
- View project details and progress.
- Update and delete projects.
- Associate projects with their members/tasks.

Tasks

- Create tasks within a project.
- Assign tasks to team members.
- Set a task's title, description, priority, and status.
- Update and delete tasks.
- Track task progress.

Task status & Priority

- Status: To Do -> In progress -> Done
- Priority: Low -> Medium -> High

Tasks

Complete up to the level you're comfortable with, do as much as you can:

Level 1: Core Full-Stack App

- Set up a **backend server** (Node/Express, or any framework you're comfortable with) and a **database** (SQL or NoSQL, your choice).
- Set up a **frontend app** (React, Vue, or any framework you're comfortable with) that talks to the backend via **fetch/axios**.
- Define **data models/schema** for Users, Projects, and Tasks.
- Backend: **CRUD** endpoints for Projects and Tasks.
- Frontend: **Main application pages** (Projects list, Project details, Tasks, Task details, ...), **navigation**, **forms** for creating/editing **Projects and Tasks**, wired to the real **backend** endpoints.
- **Task status** (To Do / In Progress / Done) and **priority** (Low / Medium / High).
- Basic validation on both frontend (form-level) and backend (API-level).
- Clean project structure on both ends (routes/controllers/models on backend; components/pages on frontend).
- No auth required yet, data can be open/unassigned to a user at this level.

Level 2: Adding Real Authentication

Extend Level 1 with:

- **Real authentication (backend):**
 - **POST /register:** create a user with a hashed password (bcrypt or similar). Never store plain-text passwords.
 - **POST /login:** verify credentials, return a JWT (or session token) on success.
 - **Protect relevant routes with authentication middleware:** reject requests without a valid token (401 Unauthorized).
- **Auth UI (frontend):** Sign up, login, and logout screens, wired to the real endpoints above. Store the token (e.g. in localStorage) and attach it to authenticated requests. Being "logged in" persists across refresh.
- **Protected routes (frontend):** redirect to Login if there's no valid token.

- **Ownership enforcement (backend):** a user can only update/delete Projects/Tasks they own or are a member of.
- **Task assignment:** assign tasks to a valid member of the project (assignedTo, a userId that must exist and belong to that project), with a UI to pick from project members.
- Loading, error, and empty states on the frontend (e.g. invalid login, empty required fields, "No projects yet").
- Responsive UI (usable on mobile and desktop widths).
- Environment variables via a .env file on the backend (DB connection string, JWT secret, port), include a file like .env.sample or show how to configure these vars in README file.

Level 3: Advanced Full-Stack Applications

Extend Level 2 with:

- **Role-based access, enforced on both ends (two roles only):**
 - **Owner:** the user who created the project. Can edit/delete the project, add/remove members, and edit/delete any task in it.
 - **Member:** a user added to the project. Can view the project and update the status of tasks assigned to them, but cannot edit the project or delete tasks.
 - **Backend:** middleware returns 403 Forbidden for unauthorized actions.
 - **Frontend:** UI reflects the user's role (hide/disable actions they can't perform).
- **User-specific content:** "My Projects" and "My Tasks" views, filtered by the logged-in user's ID.
- **Advanced filtering/searching/sorting** on Projects and Tasks (status, priority, assignee, date), implemented on the backend (query params) and reflected in the frontend UI.
- **Pagination** on list endpoints and corresponding frontend UI (e.g. "load more" or page controls).
- **Robust error handling** across the stack (backend returns consistent error shapes; frontend displays them gracefully, no crashes).
- **Notifications/feedback** on the frontend (e.g. toast on "Task created", "Project deleted").
- **Rate limiting** on sensitive backend endpoints (e.g. login).
- **Better state management** on the frontend (e.g. Context API, Zustand, or Redux) and better code organization on the backend.

- **Basic tests** for at least the core backend endpoints (auth, CRUD).

General Instructions

Submission

- Submission is via a GitHub public repository.
- A **README.md** is required and should include: project overview, technologies used, setup instructions, how to run the project, database setup/migration steps, API documentation/endpoints list, database design, and any assumptions or additional features implemented.
- Deploy your project (e.g. Vercel, Netlify, or Railway) and include the live links in the README. If for any reason it's not deployed, the README must clearly explain how to run it locally instead.
- A short explanation video (*5-10 mins*) is required, walk through your app and briefly explain your code structure, it must be submitted via Google Drive (*Make sure it's public*).
- *Send an email with the subject "Software team Task Inquiry", including: Github repo link, Deployed Links, .env files and your name.*

General Rules

- Searching online and using external resources (docs, Stack Overflow, tutorials) is allowed.
- You can use any framework you are comfortable with.
- Using AI tools to generate code is not allowed. The implementation must be your own understanding and your own work, you may be asked to explain or modify any part of your code in person, and AI-generated code you can't explain will be treated as a violation.
- The project should be completed within 5 days.
- Commit history will be reviewed: commit incrementally, not as one giant commit.
- Seed the app with mock/sample data (users, projects, tasks) so it can be viewed and tested immediately in the demo video or by reviewers, don't submit with empty data.

- Be prepared for a follow-up Q&A / code walkthrough after submission. You may be asked to explain any part of your implementation, justify design decisions, or make a small live change to your code.

For any questions during the task, contact us via **sda.curt@gmail.com**.